

Scribed by:

1. Aditi (2022MT11298)
2. Aditya Raj (2022MT11953)
3. Kinjal Anchhara (2021MT60959)
4. Suhani Soni (2022MT61981)
5. Syna Rajvanshi (2022MT1974)

1 Overview

In the last lecture, we introduced complexity theory and defined the running time for Deterministic Turing Machines (DTMs) and Non-Deterministic Turing Machines (NDTMs). We showed that a k -tape DTM can be simulated by a single-tape DTM with a polynomial slowdown ($O(t^2(n))$), while an NDTM can be simulated by a DTM with an exponential slowdown ($2^{O(t(n))}$). We defined the complexity classes $TIME(t(n))$, P, $NTIME(t(n))$, and NP. We also introduced the verifier-based definition of NP.

In this lecture, we will formally prove the equivalence of the two definitions of NP. We will then properly define and discuss the P vs NP problem, polynomial-time reductions, and the concepts of NP-hard and NP-complete languages.

2 The Class NP and P vs NP

2.1 Equivalence of NP Definitions

In the last lecture, we introduced two definitions for the class NP. The first was based on Non-Deterministic Turing Machines, and the second was based on verifiers. We will now formally prove that these two definitions are equivalent.

First, let's formally state the verifier-based definition.

Definition 1 (Polynomial-Time Verifier). A **verifier** for a language L is a deterministic algorithm (DTM) V such that:

$$L = \{x \mid \exists c \text{ such that } V(\langle x, c \rangle) \text{ accepts}\}$$

The string c is called a **certificate** or proof of membership.

V is a **polynomial-time verifier** if it runs in time polynomial in the length of x , i.e., $O(|x|^k)$ for some constant k .

Definition 2 (The Class NP - Verifier Definition). **NP** is the class of all languages that have a polynomial-time verifier.

Theorem 3. A language L is in NP if and only if L is decided by some polynomial-time Non-Deterministic Turing Machine (NDTM).

Proof. We prove both directions of the implication.

($\Rightarrow$) **Assume L has a poly-time verifier V . Construct a poly-time NDTM N for L .** Let V be a DTM that verifies L in $O(|x|^k)$ time. We construct an NDTM N that decides L as follows:

1. On input x of length n , N non-deterministically guesses a certificate string c . We know the certificate c for V must be of polynomial length in n , say $O(n^j)$. N guesses a string c of this length.
2. N then runs the deterministic verifier V on the input $\langle x, c \rangle$.
3. If V accepts, N accepts. If V rejects, N rejects that computation branch.

If $x \in L$, there exists a certificate c that V will accept. N will find this path and accept. If $x \notin L$, no certificate c exists that V will accept, so all branches of N 's computation will reject. The total running time of N is the time to guess c (poly) plus the time to run V (poly). Thus, N is a polynomial-time NDTM.

($\Leftarrow$) **Assume L is decided by a poly-time NDTM N . Construct a poly-time verifier V for L .** Let N be an NDTM that decides L in time $O(n^k)$. We construct a deterministic verifier V as follows:

1. V takes as input a pair $\langle x, c \rangle$, where x is the input string and c is a potential certificate.
2. We define the certificate c to be an encoding of an accepting computation path of N on input x . Since N runs in $O(n^k)$ time, the length of any computation path is polynomial in $n = |x|$.
3. V deterministically simulates N on input x , following the sequence of non-deterministic choices specified by the path c .
4. If the simulation of N reaches an accepting state, V accepts $\langle x, c \rangle$. Otherwise, V rejects.

If $x \in L$, there exists an accepting computation path c for N . When V is given this $\langle x, c \rangle$, it will verify it and accept. If $x \notin L$, no such accepting path exists, so V will never accept. The simulation of one step of N takes polynomial time on V . Since the path c has polynomial length $O(n^k)$, the total time for V to run the simulation is polynomial in $n = |x|$. Thus, V is a polynomial-time verifier. $\square$

2.2 The Classes P and NP

Now that we have established the models of computation, we can formally define the major complexity classes.

Definition 4 (The Class P). *The class **P** (Polynomial Time) is the set of all languages that are decidable by a **deterministic** single-tape Turing machine in polynomial time.*

$$P = \bigcup_{k \geq 1} \text{TIME}(n^k)$$

Definition 5 (The Class NP). *The class **NP** (Nondeterministic Polynomial Time) is the set of all languages that are decidable by a **non-deterministic** Turing machine in polynomial time.*

$$NP = \bigcup_{k \geq 1} \text{NTIME}(n^k)$$

The P vs NP Problem A direct consequence of these definitions is that $P \subseteq NP$. This is because any deterministic TM is, by definition, a non-deterministic TM that simply has only one choice at every step.

The most famous open question in computer science is whether this containment is proper:

$$\text{Is } P = NP?$$

Most researchers believe that $P \neq NP$, but no proof is known. To prove $P \neq NP$, one would have to find a language that is in NP but is provably not in P. To prove $P = NP$, one would have to show that every problem in NP can be solved by a polynomial-time deterministic algorithm.

To discuss this further, we need the concept of "hardness" and "completeness", which are defined using reductions.

3 Reductions and NP-Completeness

3.1 Polynomial-Time Reductions

To classify the "hardness" of problems in NP, we use the concept of a polynomial-time reduction. Informally, problem A is "reducible" to problem B if we can use a solver for B to solve A.

Definition 6 (Polynomial-Time Computable Function). *A function $f : \Sigma^* \rightarrow \Sigma^*$ is **polynomial-time computable** if there exists a polynomial-time DTM M that, given any input x on its tape, halts with $f(x)$ on its tape in $O(|x|^k)$ steps.*

Definition 7 (Polynomial-Time Reduction). *A language A is **polynomial-time reducible** to a language B , denoted $A \leq_p B$, if there is a polynomial-time computable function f such that for every $x \in \Sigma^*$:*

$$x \in A \iff f(x) \in B$$

*The function f is called the **reduction**. It transforms any instance x of problem A into an instance $f(x)$ of problem B .*

This reduction is useful because it implies that if B is "easy" (in P), then A must also be "easy".

Theorem 8. *If $A \leq_p B$ and $B \in P$, then $A \in P$.*

Proof. We can build a polynomial-time DTM for A (call it M_A) using the polynomial-time DTM for B (call it M_B) and the polynomial-time computable function f .

1. On input x , M_A first computes $y = f(x)$. This takes polynomial time, say $O(n^k)$, since f is poly-time computable.
2. M_A then runs M_B on the resulting string y . Since $B \in P$, this also takes polynomial time in the length of y , say $O(|y|^j)$.
3. Since $|y|$ is at most polynomial in $|x|$ (as f runs in poly-time), the total time is $O(n^k + (n^k)^j)$, which is still polynomial in $n = |x|$.
4. By the definition of the reduction, M_A will accept x if and only if M_B accepts y , which is true if and only if $x \in A$. Thus, M_A decides A in polynomial time, so $A \in P$.

□

3.2 NP-Hard and NP-Complete

We can now define the hardest problems in NP.

Definition 9 (NP-Hard). *A language L is **NP-hard** if every language $L' \in NP$ is polynomial-time reducible to L .*

$$\forall L' \in NP, \quad L' \leq_p L$$

An NP-hard problem is at least as hard as every problem in NP.

Definition 10 (NP-Complete). *A language L is **NP-complete** if:*

1. $L \in NP$ (it's in the class), and
2. L is NP-hard (it's one of the hardest problems in the class).

NP-complete problems are the "hardest" problems in NP. The significance of this class is captured by the following theorem.

Theorem 11. *If any NP-complete language L is in P , then $P = NP$.*

Proof. Assume L is NP-complete and $L \in P$.

1. We know L is NP-hard, so by definition, for every language $A \in NP$, we have $A \leq_p L$.
2. But we are given that $L \in P$.
3. From the previous theorem (Theorem 2), if $A \leq_p L$ and $L \in P$, it implies that $A \in P$.

4. Since this holds for every $A \in NP$, it means that $NP \subseteq P$.
5. Since we already know $P \subseteq NP$, we must have $P = NP$.

□

This means that if we can find a polynomial-time algorithm for any single NP-complete problem, we have simultaneously found one for all NP problems.

The first problem to be proven NP-complete was **SAT** (the satisfiability problem), in the foundational **Cook-Levin Theorem**.

References

- [1] A. Maheshwari, M. Smid, *Introduction to Theory of Computation*, School of Computer Science, Carleton University, 2024.
- [2] P. Linz, *An Introduction to Formal Languages and Automata, 6th Edition*, Jones Bartlett Learning, 2016.